

APPENDIX A ARTIFACT APPENDIX

A. Description & Requirements

The artifact contains all the code and binaries to reproduce the paper’s claims. The mitigation is stored in `optee_shm_patch`. `ae` contains the artifact evaluation wrapper scripts. The remaining folders all pertain to ScHMuzz.

1) *How to access*: The artifact can be downloaded via hotcrp.

2) *Hardware dependencies*: x86-64 CPU with 8 cores and 16 GB of RAM running a recent Linux.

3) *Software dependencies*: Docker is required to run the artifact.

4) *Benchmarks*: None

B. Artifact Installation & Configuration

Download and unpack the artifact archive.

Then run the following command to setup:

```
./ae/ae.sh setup
```

To run experiment E4 please send us your SSH public key over hotcrp. We will add it to the authorized keys for our VPS. The VPS will be used to access our testing devices for E4 over adb.

C. Experiment Workflow

All experiments except E4 are run from the `ae/ae.sh` scripts. The script can be used to run individual experiments or all relevant sequentially.

For E4, first connect with SSH to our VPS:

```
ssh ae@65.108.89.50
```

Then follow the instructions for the experiment.

D. Major Claims

- (C1): ScHMuzz automatically detects crashes only triggerable via double fetches in TAs (Table 1, Figure 4, Figure 5). This is proven by experiment E1
- (C2): We found six double fetch vulnerabilities (Table 2). This is proven by experiment E2.
- (C3): Rust TAs are also affected by overlapping fetches (Table 4). This is proven by experiment E3.
- (C4): Teegris, Mitee, QSEE, and Kinibi support zero-copy shared memory (Table 3). This is proven by experiment E4.

E. Evaluation

1) *Experiment (E1)*: [Automatic Double Fetch Detection] [45 compute-hour + 10 human-minutes]: Runs the fuzzing campaign on the 30 harnesses TAs including all three of ScHMuzz’s stages.

[Preparation] `./ae/ae.sh setup`

[Execution] `./ae/ae.sh e1_automatic_df_detection`

[Results] The script prints the resulting data in the same format as Table 1 and generates the coverage graph figures (Figure 4,5). Compare the results from the table to Table 1 in the paper, the numbers should be similar. Compare the coverage graphs with Figure 4 and 5 from the paper, again

they should be similar. By default this eval reduces fuzzing times to allow it to run on a commodity machine, the numbers will not be the same as in the paper.

2) *Experiment (E2)*: [TOCTTOU Vulnerabilities] [<1 compute-hour + 5 human-minutes]: Runs the proof of concepts for the six discovered vulnerabilities against the emulator until the crashes reproduce.

[Preparation] None

[Execution] `./ae/ae.sh e2_vulns`

[Results] The script runs until it triggers the crashes for each vulnerability. Output should be that all the vulnerabilities reproduce.

3) *Experiment (E3)*: [Rust TAs] [3 compute-hours + 5 human-minutes]: Runs ScHMuzz on the five Rust TAs.

[Preparation] None

[Execution] `./ae/ae.sh e3_rust`

[Results] After finishing the script prints the data in the format of Table 4. Compare the results, for 3 out of the 5 TAs ScHMuzz should detect overlapping fetches.

4) *Experiment (E4)*: [On-device Shared Memory] [<1 compute-hour + 5 human-minutes]: runs pocs on-device that trigger double fetches to confirm zero-copy shared memory.

[Preparation] Obtain access to the VPS and login via SSH.

[Execution] `cd oversharing_ae`

`./ae/ae_ondevice.sh`

[Results] The script prints the results in the same format as Table 3.

F. Customization

The `ae.sh` script can be customized using a number of environment variables.

`AE_SCALE=paper` uses the same times as the paper’s evaluation.

`AE_JOBS` scales the number of parallel emulator containers.

`AE_EXPLORE_TIME` scales the time to fuzz each TA in the exploration phase.

`AE_FAF_TIME` scales the time spent on fetch anchored fuzzing per snapshot.

G. Notes

The reproduction for C4 includes only a subset of the testing devices used for the experiment in the paper. This is because some of these devices are currently being used for other projects. Additionally, both our mitee testing devices were unrooted during an update and can not be easily rerooted.